DIA 12

DevOps e Deploy

Ambientes reproduzíveis, pipeline confiável e deploy com recuperação

Treinamento prático com decisões técnicas, código executável e critérios de qualidade

Roteiro da aula

01	**Ambiente reproduzível**	Docker e Sail com versões controladas
02	**Integração contínua**	Pipeline que bloqueia regressões antes do merge
03	**Deploy seguro**	Artefato, migração, filas, health check e rollback
04	**Projeto do dia**	Publicar uma versão com evidência de execução

Falha de deploy no ClinicFlow

A versão passou localmente, mas a produção iniciou com migration pendente, workers antigos e assets incompatíveis.

IMPACTO

- agenda retorna erro 500
- jobs executam código antigo
- rollback não reverte o banco

PERGUNTAS

- o mesmo artefato foi validado?
- a migration permite convivência?
- como detectar e recuperar rapidamente?

Ambiente local com Sail

Sail oferece uma interface previsível sobre Docker Compose. Fixe versões e mantenha os comandos essenciais no repositório.

```
./vendor/bin/sail up -d
./vendor/bin/sail composer install
./vendor/bin/sail artisan migrate --seed
./vendor/bin/sail npm ci
./vendor/bin/sail test
```

Contrato local

- mesma versão de PHP
- serviços declarados
- setup repetível
- dados de teste controlados

Um novo integrante deve executar o projeto sem reconstruir o ambiente por tentativa e erro.

Pipeline mínimo no GitHub Actions

O pipeline deve reproduzir as validações exigidas para o merge, com dependências bloqueadas e banco isolado.

```
steps:
  - uses: actions/checkout@v4
  - uses: shivammathur/setup-php@v2
    with:
      php-version: '8.3'
      coverage: none
  - run: composer install --no-interaction --prefer-dist
  - run: cp .env.example .env && php artisan key:generate
  - run: php artisan test --parallel
```

Falha o merge

- testes quebrados
- análise estática
- formatação divergente
- build de assets inválido

Cache acelera o pipeline, mas não deve esconder dependências ausentes.

Forge e Vapor

Forge

Provisiona e administra servidores. O time controla runtime, workers, scheduler, rede e capacidade.

- bom para infraestrutura previsível
- controle operacional direto
- exige manutenção do servidor

Vapor

Publica Laravel em serviços gerenciados da AWS com arquitetura serverless e configuração por ambiente.

- escala elástica
- menos gestão de servidor
- custos e limites dependem da carga

A escolha depende de carga, competências da equipe, requisitos operacionais e custo total.

Script de deploy idempotente

O deploy automatizado deve falhar cedo, ativar a nova versão de forma atômica e reiniciar processos que mantêm código em memória.

```
set -e

php artisan down --retry=20
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan queue:restart
php artisan up
```

Antes de executar

- backup verificável
- health check definido
- rollback testado
- janela de impacto conhecida

Para zero downtime, use releases versionadas e troca atômica do diretório ativo.

Migrações compatíveis com versões

Código antigo e novo podem coexistir durante o rollout. Mudanças destrutivas exigem etapas separadas.

```
Schema::table('appointments', function (Blueprint $table) {
    $table->string('external_reference')->nullable()->index();
});

Appointment::query()
    ->whereNull('external_reference')
    ->chunkById(500, fn ($items) => BackfillReference::dispatch($items->pluck('id')));
```

Sequência segura

- adicionar sem quebrar
- publicar código compatível
- preencher dados
- remover legado depois

Nunca dependa de um rollback de código para desfazer automaticamente uma migração destrutiva.

Workers e scheduler no deploy

Filas e tarefas agendadas fazem parte da versão. O deploy precisa coordenar processos de longa duração.

```
php artisan queue:restart
php artisan horizon:terminate
php artisan schedule:list
php artisan queue:monitor redis:default --max=500
```

Verifique

- supervisor reinicia workers
- jobs aceitam retry
- scheduler roda uma vez
- fila não cresce após deploy

Workers antigos podem continuar executando contratos que o código novo já alterou.

Health check, rollback e observação

O deploy termina quando a aplicação prova que está saudável, não quando o upload termina.

Health check

- HTTP 200 em endpoint dedicado
- banco e cache acessíveis
- worker processando fila

Rollback

- artefato anterior disponível
- configuração versionada
- decisão com limite de tempo

Observação

- erros e latência após release
- falhas de jobs
- identificador da versão nos logs

Defina o sinal que interrompe o rollout antes de precisar dele.

Projeto do dia: pipeline e deploy controlado

Missão

Padronizar o ClinicFlow e demonstrar um deploy reproduzível com recuperação definida.

Entregas

- Sail funcional e documentado
- workflow com testes e build
- script de deploy versionado
- health check e rollback descritos

Aceite

- pipeline falha em regressão
- migration permite convivência
- workers recebem nova versão
- evidência antes e depois do deploy

A apresentação da equipe deve ligar problema, decisão, evidência e resultado.

Checklist de entrega

1	**Reproduzir**	ambiente criado com comandos documentados
2	**Validar**	testes, análise estática e build no pipeline
3	**Publicar**	artefato imutável e configuração externa
4	**Migrar**	mudança compatível e observável
5	**Recuperar**	rollback executável e critérios claros

Saída esperada: uma versão publicada com evidências e plano de recuperação.